

Persistent Mesh-Proxy Wind Response Pipeline for 3DGS

Module Order, Operations, Effects, and Runtime Flow
Pipeline record for *Wind-Driven Deformable 3D Gaussian Splatting*

Prepared as a repo-level pipeline note

May 12, 2026

Pipeline name. Persistent Mesh-Proxy Wind Response Pipeline for 3DGS.

One-line summary. Given a trained 3DGS asset, construct a fixed-topology mesh-proxy scaffold, bind each Gaussian persistently to the proxy, compute wind loads on fixed material-space load samples, solve proxy dynamics, transport mesh deformation back to Gaussian attributes, and render the dynamic 3DGS response.

Central loop.

3DGS → simulation proxy → wind dynamics
→ Gaussian attribute transport → 3DGS rendering.

1 Purpose of This Note

이 문서는 현재 작성된 아이디어 스케치를 기준으로, 입력 3DGS 가 들어온 뒤 최종 response, 즉 dynamic rendered frame 또는 video 가 출력될 때까지 필요한 모듈을 순서대로 정리한다. 목적은 contribution 문장을 다시 쓰기 전에, 실제 pipeline dependency 를 명확히 하는 것이다. 중요한 원칙은 다음과 같다.

- 모든 모듈이 runtime 에 매 프레임 실행되는 것은 아니다. Asset setup, training/preprocessing, runtime module 을 분리해야 한다.
- 논문의 중심은 모든 구성요소의 단순 조합이 아니라, 3DGS 와 topology-aware wind simulation 사이의 **persistent representation bridge** 이다.
- 3DGS 는 최종 input 과 rendering representation 으로 남고, mesh proxy 는 내부 simulation scaffold 로 쓰인다.
- anchor, load sample, Gaussian binding 은 모두 고정된 material-space correspondence 를 만들기 위한 장치이다. Runtime 에는 topology 나 binding 을 바꾸지 않는다.

2 Pipeline-Level Input and Output

2.1 Input

기본 입력은 trained 3D Gaussian Splatting asset 이다.

$$G^0 = \{g_j^0\}_{j=1}^{N_G}, \quad g_j^0 = (\mu_j^0, R_j^0, s_j^0, \alpha_j^0, c_j^0). \quad (1)$$

여기서 각 항은 단순한 이름이 아니라, trained 3DGS 가 갖고 있는 canonical rendering state 를 의미한다. 일반적인 Inria-style 3DGS PLY 에서는 scale, opacity, rotation 이 unconstrained parameter 로 저장되어 있으므로, loader 에서 물리적으로 유효한 값으로 변환해 사용한다.

항	의미	값의 계산 또는 획득 방법
μ_j^0	Canonical Gaussian center. Gaussian g_j 의 기준 시점 중심 위치이며, proxy extraction, Gaussian-to-proxy binding, covariance transport 의 기준 좌표가 된다.	PLY 의 position field, 보통 (x_j, y_j, z_j) 에서 직접 읽는다. 이 값은 trained 3DGS 의 canonical object/world coordinate 에 있으며, 이후 deformation 은 이 위치를 직접 최적화하지 않고 proxy binding 을 통해 갱신한다.
R_j^0	Canonical Gaussian orientation. Anisotropic ellipsoid 의 local axes 가 world/canonical coordinate 에서 어떤 방향을 향하는지 나타낸다.	PLY 의 $(\hat{q}_{j0}, \hat{q}_{j1}, \hat{q}_{j2}, \hat{q}_{j3})$ 또는 rot_0..3 quaternion 을 normalize 한 뒤 rotation matrix 로 변환한다: $R_j^0 = \text{Rot} \left(\frac{\hat{q}_j}{\ \hat{q}_j\ + \varepsilon} \right).$ 이 값은 covariance 계산과 triangle-local frame transport 의 초기 orientation 으로 사용된다.
s_j^0	Canonical anisotropic scale. Gaussian ellipsoid 의 세 principal-axis standard deviation 또는 radius 역할을 한다.	PLY 의 scale_0..2 는 보통 log-scale unconstrained parameter $\hat{s}_j$ 로 저장된다. 양수 scale 은 $s_j^0 = \exp(\hat{s}_j)$ 로 계산한다. 이후 covariance 는 $\Sigma_j^0 = R_j^0 \text{diag}((s_j^0)^2)(R_j^0)^\top$ 로 구성한다.
α_j^0	Canonical opacity. Rendering alpha contribution, floater filtering, Gaussian reliability, wind-axis transmittance 의 기본 opacity evidence 로 쓰인다.	PLY 의 opacity 는 보통 logit parameter $\hat{\alpha}_j$ 로 저장된다. 실제 opacity 는 $\alpha_j^0 = \frac{1}{1 + \exp(-\hat{\alpha}_j)}$ 로 계산하며, low-opacity Gaussian 은 floater likelihood 나 reliability score 에서 낮은 신뢰도를 갖는다.
c_j^0	Canonical appearance coefficients. View-dependent color 를 표현하는 SH coefficients 이며, runtime 에서는 SH residual correction 의 기준 appearance 가 된다.	PLY 의 f_dc_* 와 선택적 f_rest_* 에서 읽는다. f_dc 는 SH degree 0 color component 이고, higher-order SH 가 있으면 c_j^0 에 함께 stack 한다. Debug color 는 보통 $\text{rgb}_j^{\text{dc}} \approx \text{clip}(C_0 f_{\text{dc},j} + 0.5, 0, 1)$ 로 확인하지만, 최종 rendering 은 full SH coefficients 를 renderer convention 에 맞게 사용한다.

위 항들을 이용해 preprocessing 단계에서 다음 파생량을 만든다.

$$\Sigma_j^0 = R_j^0 \text{diag}((s_j^0)^2)(R_j^0)^\top, \quad n_j^G = \text{eigvec}_{\min}(\Sigma_j^0). \quad (2)$$

Σ_j^0 는 Gaussian covariance transport 의 기준이고, n_j^G 는 가장 얇은 축 방향으로부터 얻은 surface-normal 후보이다. 이 normal 은 proxy surface consistency, floater likelihood, local-frame binding 의 보조 evidence 로 사용된다.

추가 입력은 사용자 또는 데이터 기반 wind signal, constraint, 그리고 rendering camera sequence 이다.

$$W(x, t), \quad \mathcal{C}_{\text{user}}, \quad \Pi_t. \quad (3)$$

항	의미	값의 계산 또는 획득 방법
$W(x, t)$	공간과 시간에 따라 변하는 wind velocity field. Load sample 위치에서 effective wind 를 계산하는 입력이다.	MVP 에서는 analytic one-way field 로 둔다: $W(x, t) = W_{\text{global}}(t) + \sum_k W_{\text{source}, k}(x, t) + W_{\text{gust}}(x, t).$ 즉 global direction/strength, fan source, sinusoidal gust, turbulence noise parameter 로 계산한다. 이후 단계에서는 measured wind 또는 learned latent wind 로 대체할 수 있다.
$\mathcal{C}_{\text{user}}$	User constraint 또는 editable control field. 어떤 부분이 고정, 부착, 핸들, 높은 stiffness, crease/rib 인지 지정한다.	사용자 paint mask, profile preset, boundary selection, geodesic distance field, manually selected vertices/faces 에서 만든다. Solver 에서는 inverse mass, attachment compliance, pinned vertex mask, material stiffness field 로 변환한다.
Π_t	Rendering camera 또는 evaluation view sequence. Dynamic Gaussian state $G(t)$ 를 어떤 view 에서 render 할지 정한다.	각 frame 의 camera intrinsics/extrinsics 로 구성된다. 예를 들어 synthetic setup 에서는 JSON camera file 의 K , camera-to-world 또는 world-to-camera matrix 에서 읽는다. Training/evaluation 에서는 ground-truth view 와 같은 camera path 를 사용하고, preview 에서는 turntable 또는 fixed camera 를 사용한다.

2.2 Output

최종 출력은 time-varying Gaussian asset 과 rendered response 이다.

$$G(t) = \{g_j(t)\}_{j=1}^{N_G}, \quad I_t = \mathcal{R}_{3\text{DGS}}(G(t), \Pi_t). \quad (4)$$

영상 출력이면:

$$\mathcal{I} = \{I_1, I_2, \dots, I_T\}. \quad (5)$$

내부 상태로는 deformed proxy mesh 가 존재한다.

$$M(t) = (V(t), E, F). \quad (6)$$

$M(t)$ 는 사용자가 직접 보는 output 은 아니지만, wind deformation, boundary condition, force propagation, Gaussian transport 를 연결하는 핵심 상태이다.

3 Phase Separation

전체 pipeline 은 세 phase 로 나누는 것이 가장 명확하다.

Phase	실행 시점	포함 모듈
Asset setup	asset 당 1 회	3DGS preprocessing, proxy extraction, simulation proxy construction, anchor/load sample setup, Gaussian binding
Training / pre-processing	dataset 또는 asset 준비 중	Blender dense supervision, dense-to-proxy force projection, projection-risk-based load sample refinement, force/appearance model training
Runtime response	매 frame	wind evaluation, exposure estimation, force-space load prediction, load-to-anchor projection, XPBD solver, Gaussian attribute transport, SH correction, rendering

핵심적으로, **runtime** 에는 **remeshing**, **Gaussian rebinding**, **load sample 재배치가 없다**. Runtime 에서는 고정된 topology, 고정된 binding, 고정된 material-space samples 위에서 state 와 force value 만 갱신한다.

4 Module Order Overview

순서	모듈	실행 시점	핵심 역할
1	3DGS preprocessing	Asset setup	floater 완화, surface-oriented attribute 와 Gaussian reliability 후보 계산
2	Proxy mesh extraction	Asset setup	3DGS 에서 raw topology scaffold 생성
3	Simulation-oriented proxy construction	Asset setup	render mesh 가 아니라 compact physics mesh 구성
4	Anchor and load sample setup	Asset setup	simulation anchors 와 fixed material-space load samples 생성
5	Gaussian-to-proxy binding	Asset setup	Gaussian 을 triangle id, barycentric coordinate, local frame 에 persistently binding
6	Blender dense supervision	Training only	dense force, deformation, rendered reference 생성
7	Dense-to-proxy force projection	Training / pre-processing	dense force 를 proxy/load target 으로 압축하면서 net force 와 wrench 보존
8	Runtime wind field evaluation	Runtime	현재 load sample 위치에서 wind 계산
9	Wind exposure estimation	Runtime	Gaussian/proxy opacity 기반 exposure 와 effective wind 계산
10	Force-space load prediction	Runtime	deformation state 가 아니라 aerodynamic load 예측

순서	모듈	실행 시점	핵심 역할
11	Load-to-anchor projection	Runtime	load sample force 를 simulation vertices 로 누적
12	Proxy dynamics solver	Runtime	XPBD/PBD 로 topology-aware deformation 계산
13	Gaussian attribute transport	Runtime	mesh deformation 을 Gaussian mean/covariance/local frame 으로 전달
14	SH / appearance correction	Runtime	analytic SH correction 과 residual 로 appearance 보정
15	3DGS rendering output	Runtime	updated Gaussian set 을 standard 3DGS renderer 로 출력

5 Module 1: 3DGS Preprocessing

Input

$$G^0 = \{g_j^0\}_{j=1}^{N_G}, \quad g_j^0 = (\mu_j^0, R_j^0, s_j^0, \alpha_j^0, c_j^0). \quad (7)$$

이 식에서 G^0 는 wind deformation 을 적용하기 전의 전체 trained 3DGS object 또는 asset 을 의미한다. g_j^0 는 그 object 를 구성하는 j 번째 Gaussian splat 하나이다. 따라서 G^0 는 하나의 연속 surface mesh 가 아니라, N_G 개의 Gaussian state 가 모인 집합으로 본다.

각 g_j^0 는 독립적인 작은 rendering primitive 의 canonical state 이다. μ_j^0 는 중심 위치, R_j^0 는 local ellipsoid axes 의 방향, s_j^0 는 anisotropic scale, α_j^0 는 opacity, c_j^0 는 color 또는 SH appearance coefficients 를 나타낸다. 위첨자 0 는 이 값들이 deformed state 가 아니라 rest/canonical state 에서 읽힌 값을 뜻한다. Runtime wind response 에서는 이 per-Gaussian state 가 proxy binding 을 통해 $g_j(t)$ 로 갱신된다.

[Ref] 이 입력 표현은 기존 3D Gaussian Splatting 의 표준 state representation 에 기반한다 [3]. 논문 작성 시에는 이 식 자체를 novelty 로 주장하지 않고, trained 3DGS asset 을 읽는 기본 형식으로 둔다. 우리의 관심은 이후 단계에서 이 state 를 wind-oriented proxy construction 과 persistent binding 의 evidence 로 어떻게 사용하는지에 있다.

Operation

먼저 각 Gaussian 의 anisotropic shape 을 하나의 matrix 로 표현하기 위해 Gaussian covariance 를 계산한다. s_j^0 는 local ellipsoid axes 방향의 scale 이고, R_j^0 는 그 local axes 를 canonical/world coordinate 로 회전시키는 orientation 이다. 따라서 diagonal covariance 를 만든 뒤 R_j^0 로 회전시키면, Gaussian g_j^0 가 3D 공간에서 실제로 어떤 방향으로 길고 얇게 퍼져 있는지 표현할 수 있다.

$$\Sigma_j^0 = R_j^0 \text{diag}((s_j^0)^2)(R_j^0)^\top. \quad (8)$$

[Ref] 이 covariance 계산은 Gaussian scale 과 rotation 에서 covariance 를 구성하는 표준적인 Gaussian parameter conversion 이다. 3DGS 는 anisotropic covariance 를 갖는 3D Gaussians 를 scene primitive 로 사용하므로, 이 구성은 기존 3DGS 표현에 기반한다 [3]. 이후 transport 의 기준으로 쓰이지만, covariance 구성 자체는 기존 Gaussian 수학 및 3DGS 표현에 속한다.

여기서 scale 을 제공하는 이유는 covariance 가 standard deviation 자체가 아니라 variance 를 담은 matrix 이기 때문이다. 이 Σ_j^0 는 이후 Gaussian covariance transport 의 기준이 된다. 즉 proxy mesh 가 휘거나 회전할 때 Gaussian center 만 옮기는 것이 아니라, Gaussian ellipsoid 의 방향과 퍼짐도 surface deformation 을 따라가도록 만들기 위한 초기 shape descriptor 이다.

다음으로 가장 작은 scale 방향을 Gaussian normal 후보로 사용할 수 있다. Surface 근처의 anisotropic Gaussian 은 보통 surface tangent 방향으로 넓게 퍼지고, surface normal 방향으로 얇게 퍼진다. 따라서 covariance 의 가장 작은 eigenvalue 에 해당하는 eigenvector 는 이 Gaussian 이 암시하는 얇은 방향, 즉 surface normal 후보로 해석할 수 있다.

$$n_j^G = \text{eigvec}_{\min}(\Sigma_j^0). \quad (9)$$

[Ref] + [Ours] Gaussian orientation 과 surface alignment 를 활용하는 관점은 SuGaR 나 GOF 같은 surface-aware Gaussian/mesh extraction 계열 연구와 연결된다 [2, 4]. 다만 여기서는 n_j^G 를 확정 normal 로 주장하지 않고, proxy extraction 과 Gaussian-to-proxy binding 을 안정화하기 위한 보조 evidence 로 쓰는 adaptation 이다.

다만 n_j^G 는 확정적인 mesh normal 이 아니라 Gaussian 에서 얻은 후보 normal 이다. 거의 isotropic 한 Gaussian, noisy Gaussian, floater Gaussian 에서는 이 방향이 불안정할 수 있고, eigenvector 의 부호도 양/음이 모호하다. 그래서 이 값은 proxy extraction 이나 binding 에서 surface consistency 를 판단하는 보조 evidence 로 사용하고, 이후 proxy mesh normal 또는 local frame 과 함께 검증한다.

마지막으로 opacity reliability 를 계산한다. Opacity α_j^0 가 낮은 Gaussian 은 실제 surface 를 안정적으로 설명하기보다 floater, background noise, weak contribution 일 가능성이 상대적으로 크다. 하지만 opacity 의 절대값 분포는 asset 마다 다르기 때문에 고정 threshold 를 쓰지 않고, asset 내부의 opacity percentile 을 이용해 정규화한다.

$$q_j^\alpha = \text{smoothstep}(P_{\text{low}}(\alpha^0), P_{\text{high}}(\alpha^0), \alpha_j^0). \quad (10)$$

[Heuristic] + [Ours] 이 항은 표준 3DGS 식이라기보다 asset-wise opacity 분포 차이를 줄이기 위한 robust heuristic 이다. q_j^α 는 본 논문의 main contribution 이 아니라, proxy extraction 과 binding 에서 unreliable Gaussian 의 영향력을 낮추기 위한 supporting design 으로 둔다.

$P_{\text{low}}(\alpha^0)$ 와 $P_{\text{high}}(\alpha^0)$ 는 전체 asset 의 canonical opacity set $\{\alpha_j^0\}_{j=1}^{N_G}$ 에서 계산한 low/high percentile threshold 이다. 이 percentile normalization 의 목적은 asset 마다 달라지는 opacity scale 에 대해 robust 한 상대 신뢰도를 만드는 것이다. 따라서 특정 숫자 20/80 자체가 물리 법칙이나 3DGS 이론에서 고정되는 값은 아니며, MVP 에서 사용할 수 있는 heuristic default 로 본다. 초기 구현에서는

$$P_{\text{low}} = P_{20}, \quad P_{\text{high}} = P_{80}$$

로 시작하고, 이후 P_{10}/P_{90} , P_{20}/P_{80} , P_{30}/P_{70} 같은 ablation 으로 proxy extraction 과 binding 안정성에 미치는 영향을 확인한다. smoothstep 은 낮은 opacity 영역을 0 에 가깝게, 높은 opacity 영역을 1 에 가깝게 부드럽게 보낸다. 따라서 q_j^α 는 Gaussian 을 즉시 삭제하기 위한 binary mask 가 아니라, 이 Gaussian 이 surface evidence 로 얼마나 믿을 만한지 나타내는 continuous reliability weight 이다. MVP 에서는 이 값을 proxy extraction, binding score, floater likelihood 에서 공통 confidence 로 사용할 수 있다.

Output

$$\tilde{G}^0 = \{(g_j^0, q_j^\alpha, n_j^G)\}_{j=1}^{N_G}. \quad (11)$$

[Ours] $\tilde{G}^0$ 로 원본 state 와 보조 evidence 를 묶어 넘기는 것은 pipeline-specific design 이다. 이는 새 rendering representation 을 제안하는 것이 아니라, wind-oriented proxy extraction 과 persistent Gaussian-to-proxy binding 을 안정화하기 위한 preprocessing package 이다.

$\tilde{G}^0$ 는 원본 3DGS asset 을 바꾼 새로운 rendering asset 이라기보다, preprocessing 을 통해 보조 정보를 붙인 enriched Gaussian set 이다. 각 Gaussian 은 원래 state g_j^0 를 그대로 유지하면서, reliability q_j^α 와 Gaussian normal candidate n_j^G 를 추가로 갖는다. 이 출력은 다음 단계인 proxy mesh extraction 과 Gaussian-to-proxy binding 에서 사용된다.

Effect

이 단계의 효과는 raw 3DGS 를 곧바로 mesh extraction 이나 binding 에 넣지 않고, 각 Gaussian 이 얼마나 surface-like 한지와 어떤 local orientation 을 암시하는지 먼저 평가한다는 점이다. Low-opacity floater 나 noisy Gaussian 은 proxy extraction 과 binding 을 망칠 수 있으므로 q_j^α 를 통해 영향력을 낮춘다. 반대로 신뢰도 높은 anisotropic Gaussian 은 covariance 와 normal 후보를 통해 surface orientation 의 단서를 제공한다.

주의할 점은 preprocessing 이 aggressive pruning 단계가 아니라는 것이다. 특히 isolated Gaussian 을 직접 제거하면 leaf tip, antenna-like thin structure, sparse valid detail 처럼 실제로 의미 있는 얇은 구조가 사라질 수 있다. 따라서 isolation 은 단독 제거 조건으로 쓰지 않고, low opacity, surface distance, user protection score 등과 함께 gated floater likelihood 에서만 사용한다.

6 Module 2: Proxy Mesh Extraction

Input

$$\tilde{G}^0. \quad (12)$$

$\tilde{G}^0$ 는 Module 1 에서 원본 Gaussian state 에 reliability 와 normal candidate 를 붙인 enriched Gaussian set 이다. 이 모듈은 그 Gaussian 집합을 바로 simulation 에 쓰지 않고, 먼저 connected surface representation 으로 바꾸는 단계이다. 입력 3DGS 는 point-like Gaussian 들의 집합이므로 edge, face, boundary, geodesic distance 같은 topological relation 이 명시적으로 존재하지 않는다.

Operation

3DGS 에서 raw surface 또는 mesh proxy 를 추출한다.

$$\tilde{G}^0 \rightarrow M_{\text{raw}} = (V_{\text{raw}}, E_{\text{raw}}, F_{\text{raw}}). \quad (13)$$

M_{raw} 는 extraction method 가 만든 초기 mesh 이다. V_{raw} 는 vertex positions, E_{raw} 는 edges, F_{raw} 는 triangular faces 를 의미한다.

본 프로젝트의 기본 선택은 GOF-style extraction 을 main raw proxy extraction 으로 두는 것이다 [4]. GOF 는 trained 3DGS asset 에서 opacity field 를 통해 surface 를 복원하는 방향이므로, 현재 pipeline 의 입력을 3DGS 그대로 유지하면서 mesh proxy 를 얻기에 가장 자연스럽다. 따라서 real trained 3DGS asset 에 대한 기본 경로는 다음처럼 둔다.

$$\tilde{G}^0 \xrightarrow{\mathcal{E}_{\text{GOF}}} M_{\text{raw}}. \quad (14)$$

[Ref] + [Ours] GOF-style extraction 자체는 기존 3DGS-to-surface reconstruction baseline 으로 사용한다. 우리의 novelty 는 GOF mesh extraction 알고리즘을 새로 제안하는 데 있지 않고, 추출된 rendering-oriented raw mesh 를 wind simulation 을 위한 compact proxy 로 바꾸고, 그 위에 persistent Gaussian binding, fixed load samples, anchor-based dynamics, SH correction 을 연결하는 데 있다.

비교군으로는 SuGaR-style extraction 과 TSDF-style extraction 을 둔다. SuGaR 는 Gaussian 을 surface-aligned 하게 regularize 하고 mesh reconstruction 과 mesh rendering 을 함께 고려하므로 mesh editing 이나 surface correspondence 관점에서 강한 비교군이 될 수 있다 [2]. 다만 추가 regularization/refinement 과정이 필요하고, thin open structure 나 leaf-like boundary 에서는 smoothing 또는 closure artifact 가 생길 수 있으므로 main default 보다는 alternative baseline 으로 둔다. TSDF-style extraction 은 여러 view 의 rendered depth 를 volumetric field 로 fuse 한 뒤 surface 를 추출하는 단순하고 renderer-agnostic 한 baseline 이다 [1]. 구현과 비교가 쉽다는 장점이 있지만, view coverage, depth quality, truncation

scale 에 민감하고 얇은 구조와 open boundary 를 약하게 복원할 수 있으므로 fallback 또는 lower-bound baseline 으로 둔다.

여기서 중요한 점은 mesh extraction 자체를 main novelty 로 주장하지 않는 것이다. 이 모듈의 목적은 topology-aware wind response 를 위한 internal scaffold 를 만드는 것이다. 즉 최종 rendering 은 여전히 3DGS 로 수행하지만, simulation 과 material-space correspondence 를 정의하기 위해 mesh proxy 를 중간 표현으로 둔다.

Output

$$M_{\text{raw}}. \quad (15)$$

M_{raw} 는 아직 바로 solver 에 넣기 좋은 mesh 가 아니다. Extraction 결과는 rendering surface 를 따라가도록 만들어진 경우가 많아서 triangle 크기, aspect ratio, boundary loop, non-manifold artifact, thin part sampling 이 simulation 관점에서는 불안정할 수 있다. 따라서 이 출력은 다음 Module 3 에서 physics-quality mesh 로 정리되는 중간 산출물로 본다.

Effect

proxy mesh 가 생기면 vanilla 3DGS 에 없는 edge/face connectivity, boundary loop, local surface frame, force propagation path, artist constraint site 가 생긴다. 즉, unordered Gaussian set 을 connected deformable surface 로 다룰 수 있게 된다. 이 단계가 없으면 wind force 를 Gaussian 별로 독립적으로 적용해야 하므로, 한 부분에 걸린 힘이 주변 surface 로 퍼지는 cloth-like 또는 shell-like response 를 만들기 어렵다.

Development Plan

Mesh extraction module 은 전체 pipeline 을 한 번에 완성하려고 하지 않고, 독립적으로 검증 가능한 module 로 개발한다. 초기 목표는 어떤 extraction method 를 쓰더라도 같은 interface 로 raw mesh 와 품질 리포트를 내보내는 것이다. 이를 위해 먼저 공통 data contract 를 정의한다. 입력은 trained 3DGS asset, camera metadata, extraction config 이고, 출력은 M_{raw} , per-vertex/per-face normal, extraction provenance, quality metrics 를 포함하는 raw proxy package 로 둔다.

첫 구현 단계에서는 GOF adapter 를 main path 로 만든다. GOF 실행 결과를 우리 pipeline 의 $M_{\text{raw}} = (V_{\text{raw}}, E_{\text{raw}}, F_{\text{raw}})$ 형식으로 변환하고, mesh validity, connected components, boundary loops, face aspect ratio, self-intersection 후보, Gaussian-to-surface distance histogram 을 리포트한다. 이 단계의 성공 기준은 solver 까지 가지 않고도 “3DGS asset 에서 추출한 mesh 가 Module 3 remeshing 의 입력으로 안정적으로 전달되는가” 를 확인하는 것이다.

그 다음 SuGaR adapter 와 TSDF adapter 를 같은 interface 의 comparison baseline 으로 추가한다. 두 비교군은 main pipeline 을 바꾸기 위한 것이 아니라, raw mesh 품질과 이후 binding/remeshing 안정성을 GOF 와 비교하기 위한 ablation 이다. 따라서 세 방법은 모두 동일한 downstream checker 를 통과해야 하며, 최종 비교는 rendering mesh 품질만이 아니라 simulation proxy 로 변환되기 쉬운지까지 함께 본다.

7 Module 3: Simulation-Oriented Proxy Construction

Input

$$M_{\text{raw}}, \quad \tilde{G}^0, \quad C_{\text{user}}. \quad (16)$$

여기서 M_{raw} 는 extraction 으로 얻은 초기 mesh 이고, $\tilde{G}^0$ 는 mesh 가 어떤 Gaussian detail 을 보존해야 하는지 알려주는 evidence source 이다. C_{user} 는 pin, support, crease, rib, boundary preference 처럼 사용자가 제어하고 싶은 material/constraint 정보를 담는다. 이 세

입력을 함께 쓰는 이유는 simulation mesh 가 단순히 geometric surface 만 따라가는 것이 아니라, 나중에 force propagation 과 user control 에 적합해야 하기 때문이다.

Operation

raw mesh 를 render-quality mesh 가 아니라 physics-quality mesh 로 remesh 한다. 이 remeshing 은 asset setup 에서 한 번만 수행한다.

$$M_{\text{raw}} \rightarrow M_{\text{sim}}^0 = (V^0, E, F). \quad (17)$$

M_{sim}^0 는 rest state 의 simulation proxy 이다. V^0 는 canonical vertex positions 이고, E, F 는 runtime 동안 고정되는 edge/face topology 이다. 중요한 점은 runtime 에는 remeshing 하지 않는다는 것이다. Topology 가 고정되어야 Gaussian binding, anchor index, load sample coordinate 가 시간에 따라 유지된다.

simulation target edge length 는 score field 로 정한다.

$$h_{\text{sim}}(p) = \text{clamp} \left(\frac{h_0}{\sqrt{1 + s_{\text{sim}}(p)}}, h_{\min}, h_{\max} \right). \quad (18)$$

$h_{\text{sim}}(p)$ 는 위치 p 주변에서 원하는 target edge length 이다. $s_{\text{sim}}(p)$ 가 크면 보존해야 할 구조나 binding demand 가 높다고 보고 edge length 를 작게 만든다. 반대로 score 가 낮은 완만한 영역은 더 큰 triangle 을 허용하여 solver vertex 수를 줄인다. $h_{\min}$ 과 $h_{\max}$ 는 지나치게 촘촘하거나 거친 mesh 가 생기지 않도록 막는 안전한 하한/상한이다.

MVP score 는 다음처럼 둘 수 있다.

$$s_{\text{sim}}(p) = 2.0B(p) + 1.5K_{\text{geom}}(p) + 0.7R_G(p). \quad (19)$$

이 score 는 세 종류의 요구를 섞는다. $B(p)$ 는 constraint 나 boundary 보존 요구, $K_{\text{geom}}(p)$ 는 geometry detail 보존 요구, $R_G(p)$ 는 Gaussian 을 안정적으로 transport 해야 하는 rendering demand 이다. 가중치는 초기값일 뿐이며, ablation 에서는 boundary-only, geometry-only, reliability-weighted demand 제거 등을 비교할 수 있다.

Boundary field

$$B(p) = \max_{\ell \in \mathcal{L}_{\text{bdry}}} w_{\ell} \exp \left(-\frac{d_{\text{geo}}(p, \ell)^2}{2r_B^2} \right). \quad (20)$$

이 항은 flag pole edge, curtain rail, leaf stem, pinned boundary 같은 constraint-critical region 을 보존한다. $\mathcal{L}_{\text{bdry}}$ 는 사용자가 지정하거나 자동 검출한 boundary/support curve 들의 집합이다. $d_{\text{geo}}(p, \ell)$ 는 surface 위에서 point p 와 boundary curve ℓ 사이의 geodesic distance 이고, r_B 는 boundary 영향이 퍼지는 폭이다. Euclidean distance 가 아니라 geodesic distance 를 쓰는 이유는 얇은 접힘이나 가까운 반대편 surface 가 잘못 같은 boundary 영향권으로 묶이는 것을 줄이기 위해서이다.

Geometry field

face normal variation 기반으로 계산할 수 있다.

$$K_{\text{geom}}(f) = \frac{1}{|\mathcal{N}(f)|} \sum_{f' \in \mathcal{N}(f)} \frac{1 - \langle n_f, n_{f'} \rangle}{\|c_f - c_{f'}\| + \varepsilon}. \quad (21)$$

$K_{\text{geom}}(f)$ 는 face f 주변에서 normal 이 얼마나 빠르게 변하는지 보는 curvature-like score 이다. $\mathcal{N}(f)$ 는 neighboring faces, n_f 와 c_f 는 각각 face normal 과 centroid 이다. 두 face normal 이 비슷하면 $1 - \langle n_f, n_{f'} \rangle$ 가 작고, fold 나 sharp edge 처럼 normal 이 급격히 바뀌면 값

이 커진다. 거리로 나누는 이유는 같은 normal 변화라도 짧은 거리에서 발생하면 더 날카로운 구조로 보기 위해서이다.

robust normalization 은 다음처럼 둔다.

$$\hat{K}_{\text{geom}}(p) = \text{clamp} \left(\frac{K_{\text{geom}}(p) - P_5}{P_{95} - P_5 + \varepsilon}, 0, 1 \right). \quad (22)$$

이 항은 leaf tip, cloth fold, paper edge, narrow strip 같은 thin or high-curvature structure 를 보존한다. P_5 와 P_{95} 를 쓰는 이유는 소수의 outlier curvature 가 전체 normalization 을 망치는 것을 막기 위해서이다. 실제 remeshing score 에는 normalized $\hat{K}_{\text{geom}}$ 를 넣는 것이 안전하며, 본 문서에서 K_{geom} 라고 쓸 때는 필요에 따라 normalized field 를 의미하도록 약속할 수 있다.

Reliability-weighted Gaussian transport demand

raw Gaussian density 대신, surface-consistent 하고 transport 할 가치가 있는 Gaussian 의 demand 를 사용한다.

$$R_G(p) = 1 - \exp \left(- \frac{\sum_j q_j \eta_j K_j(p)}{\tau_R} \right). \quad (23)$$

$$K_j(p) = \exp \left(- \frac{\|p - p_j\|^2}{2r_j^2} \right), \quad (24)$$

여기서 p_j 는 Gaussian center μ_j^0 를 proxy surface 에 projection 한 point 이고, r_j 는 surface footprint radius 이다. η_j 는 rendering contribution 이며, MVP 에서는 $\eta_j = 1$ 로 둘 수 있다. τ_R 는 saturation parameter 로 P_{90} 같은 percentile 을 사용할 수 있다. 이 항의 의도는 단순히 Gaussian 이 많은 곳을 중요하다고 보는 것이 아니다. Gaussian 수는 reconstruction 품질, optimizer behavior, noisy floater 에 영향을 받을 수 있으므로 raw density 만 쓰면 잘못된 곳을 과하게 보존할 수 있다. 따라서 q_j 로 reliability 를 반영하고, η_j 로 실제 rendering contribution 을 반영한 뒤, $K_j(p)$ 로 surface 위치 p 주변에 영향을 퍼뜨린다. $1 - \exp(\cdot)$ 형태는 demand 가 일정 수준 이상 커지면 saturation 되도록 하여 한 영역이 과도하게 remesh 되는 것을 막는다.

Gated floater likelihood

isolation 은 직접 reliability factor 로 쓰지 않는다. 대신 low opacity, far from surface, isolated, and not protected 인 경우에만 floater likelihood 로 작동한다.

$$q_j = q_j^{\text{base}} \exp(-\lambda_{\text{float}} p_j^{\text{float}}), \quad (25)$$

$$p_j^{\text{float}} = I_j L_j D_j (1 - S_j), \quad (26)$$

$$I_j = \exp \left(- \frac{N_j^{\text{nbr}}}{N_{\text{ref}}} \right), \quad (27)$$

$$L_j = 1 - q_j^{\alpha}, \quad (28)$$

$$D_j = 1 - q_j^{\text{surf}}, \quad (29)$$

$$q_j^{\text{surf}} = \exp \left(- \frac{d_j^2}{2\sigma_d^2} \right). \quad (30)$$

q_j^{base} 는 opacity, view contribution, surface consistency 등에서 얻은 기본 reliability 이다. p_j^{float} 는 floater 일 가능성을 나타내는 soft penalty 이고, 네 조건이 동시에 맞을 때만 커진다. I_j 는 주변 Gaussian 이 적을수록 커지는 isolation score, L_j 는 low-opacity 일수록 커지는

score, D_j 는 surface 에서 멀수록 커지는 score 이다. 마지막 $(1 - S_j)$ 는 보호되어야 하는 구조에 서는 penalty 를 끄기 위한 gate 이다. 이 곱 구조 덕분에 “isolated but valid” thin detail 을 단 순히 제거하지 않는다.

protection score 는 다음처럼 정의한다.

$$S_j = \max_{s \in \mathcal{S}_j} s, \quad (31)$$

$$\begin{aligned} \mathcal{S}_j = & \{B(p_j), K_{\text{geom}}(p_j), C_{\text{constraint}}(p_j)\} \\ & \cup \{T_{\text{thin}}(p_j) \text{ if implemented}\} \\ & \cup \{q_j^{\text{view}} \text{ if measured or approximated}\}. \end{aligned} \quad (32)$$

주의할 점은 q_j^{view} 가 없을 때 base reliability 에서는 neutral factor 로 1 을 둘 수 있지만, S_j 의 max 에는 넣지 않는다는 것이다. 그렇지 않으면 $S_j = 1$ 이 되어 floater penalty 가 항 상 꺼진다. S_j 는 boundary, high-curvature geometry, user constraint, thin-structure detector, measured view contribution 중 하나라도 강하면 Gaussian 을 보호한다. 따라서 floater filtering 은 removal rule 이 아니라 risk weighting rule 이다. 이 관점은 valid sparse detail 과 noise 를 구분해야 하는 3DGS preprocessing 에서 특히 중요하다.

Output

$$M_{\text{sim}}^0 = (V^0, E, F). \quad (33)$$

M_{sim}^0 는 이후 모든 runtime module 이 공유하는 fixed-topology proxy 이다. 이 mesh 의 vertex/face index 는 simulation anchors, load samples, Gaussian bindings 의 reference index 가 되므로 asset setup 이후에는 바뀌지 않는다.

Effect

이 모듈은 runtime remeshing 없이 사용할 compact simulation mesh 를 만든다. 결과적으로 boundary, thin structure, Gaussian binding quality, simulation stability 를 동시에 확보 한다. 렌더링 품질만 보면 원본 Gaussian 이 담당하고, dynamics 품질은 M_{sim}^0 가 담당한다. 따라서 이 모듈의 성공 기준은 mesh 가 시각적으로 가장 예쁜지가 아니라, stable solver, stable binding, controllable constraint 를 동시에 만족하는지이다.

8 Module 4: Simulation Anchors and Load Samples

Input

$$M_{\text{sim}}^0 = (V^0, E, F). \quad (34)$$

이 모듈은 이미 topology 가 정리된 simulation proxy 를 입력으로 받는다. 목표는 같은 mesh 위에서 세 종류의 고정 material-space reference 를 정의하는 것이다: solver 가 움직일 simulation anchors, 사용자가 고정하거나 제어할 constraint anchors, wind load 를 샘플링할 load samples 이다.

Operation: simulation anchors

simulation anchor 는 mesh vertex 이다.

$$A_{\text{sim}} = V. \quad (35)$$

A_{sim} 는 solver 가 실제로 상태를 적분하는 자유도 집합이다. 초기 MVP 에서는 proxy mesh vertex 전체를 simulation anchor 로 쓰는 것이 가장 단순하고 안정적이다. 나중에 더 coarse

한 anchor graph 를 쓰고 싶다면 $A_{\text{sim}} \subset V$ 로 줄일 수 있지만, 그러면 interpolation 과 force projection 이 추가로 필요하다.

각 anchor state 는 다음과 같다.

$$a_i(t) = (x_i(t), v_i(t), m_i, \theta_i). \quad (36)$$

$x_i(t)$ 와 $v_i(t)$ 는 runtime position/velocity 이고, m_i 는 vertex 에 할당된 mass 이다. θ_i 는 stiffness, damping, drag response, density 같은 material parameter 를 담는 자리이다. 이 state 는 Gaussian state 가 아니라 proxy simulation state 이다. Gaussian 은 직접 물리 solver 의 자유도가 되지 않고, 뒤의 transport module 에서 이 proxy state 를 따라간다.

area 와 mass 는 incident faces 로부터 계산한다.

$$A_i = \sum_{T \ni i} \frac{\text{area}(T)}{3}, \quad m_i = \rho A_i. \quad (37)$$

A_i 는 vertex i 가 대표하는 surface area 이다. 각 triangle area 를 세 vertex 에 균등 분배하면 간단한 lumped mass 를 만들 수 있다. ρ 는 areal density 이며, cloth, paper, rubber-like shell 등 profile preset 에 따라 바꿀 수 있다.

material parameter 는 초기에는 per-vertex field 보다 material-class scalar 로 시작하는 편이 안전하다.

$$\theta_i \approx \theta_{\text{mat}} = \{\alpha, d, \beta, \rho\}. \quad (38)$$

여기서 α 는 compliance 또는 inverse stiffness, d 는 damping, β 는 bending-related parameter, ρ 는 density 로 둘 수 있다. 초기에는 모든 vertex 에 같은 material-class scalar 를 부여하고, 이후 controllable material field 가 안정되면 per-vertex 또는 painted field 로 확장한다.

Operation: constraint anchors

사용자 pin 또는 support 는 subset 이다.

$$A_{\text{pin}} \subset A_{\text{sim}}. \quad (39)$$

A_{pin} 은 solver 자유도 중 움직임을 제한하거나 target 에 붙일 anchor subset 이다. 예를 들어 연의 뼈대, 종이비행기의 접힌선, 깃발의 pole edge, leaf stem 같은 부분은 사용자 constraint 나 profile preset 으로 지정할 수 있다. 이 constraint anchor 는 앞서 말한 simulation anchor 와 목적이 다르다. Simulation anchor 는 solver 의 계산 자유도이고, constraint anchor 는 그 자유도 중 제어 조건이 걸리는 subset 이다.

hard pin 은:

$$x_i(t) = x_i^0, \quad i \in A_{\text{pin}}. \quad (40)$$

Hard pin 은 해당 vertex 가 rest position 에서 움직이지 않는다는 의미이다. 완전히 고정된 support 나 boundary 를 표현할 때 좋지만, 너무 많이 쓰면 deformation 이 인위적으로 보일 수 있다.

soft pin 은:

$$C_i^{\text{pin}}(x) = x_i - x_i^{\text{target}}. \quad (41)$$

Soft pin 은 target 과의 차이를 constraint 로 두고 compliance 를 통해 어느 정도 따라가게 하는 방식이다. 사용자가 handle 을 움직이거나, 뼈대가 완전 고정이 아니라 약간 휘는 경우에는 hard pin 보다 soft pin 이 더 자연스럽다.

Operation: fixed material-space load samples

wind force 는 반드시 simulation vertex 에서만 계산하지 않는다. face 위에 fixed material-space load sample 을 둔다.

$$q_l = (T_l, b_l). \quad (42)$$

q_l 는 l 번째 load sample 의 material-space coordinate 이다. T_l 은 sample 이 놓인 rest-state triangle index 이고, b_l 은 그 triangle 내부의 barycentric coordinate 이다. 따라서 sample 은 월드 좌표에 고정된 점이 아니라, mesh material 에 붙어 있는 점이다. 바람에 의해 mesh 가 움직여도 같은 삼각형 위의 같은 재료 위치를 계속 따라간다.

runtime 위치는:

$$x_l(t) = \sum_{i \in T_l} b_{li} x_i(t). \quad (43)$$

이 식은 barycentric interpolation 으로 load sample 의 현재 위치를 계산한다. 중요한 점은 exposure 나 wind 가 frame 마다 바뀌어도 $q_l = (T_l, b_l)$ 자체는 바뀌지 않는다는 것이다. 즉 우리는 runtime adaptive resampling 이 아니라 fixed material-space sampling 을 사용한다. 이 선택 덕분에 force history 와 temporal correspondence 가 안정적으로 유지된다.

Output

$$A_{\text{sim}}, \quad A_{\text{pin}}, \quad Q_{\text{load}} = \{q_l\}. \quad (44)$$

A_{sim} 는 solver 자유도, A_{pin} 은 constraint 가 걸리는 자유도 subset, Q_{load} 는 wind force 를 평가할 material-space samples 이다. 세 집합은 모두 rest proxy mesh 위에서 한 번 정의되고 runtime 동안 유지된다.

Effect

이 모듈은 simulation resolution 과 force sampling resolution 을 분리한다. 따라서 wind load 가 복잡한 곳에는 load sample 만 촘촘히 둘 수 있고, XPBD solver vertex 수는 작게 유지할 수 있다. 이 구분이 본래 anchor 설계 의도와 연결된다. 모든 지점에서 비싼 force evaluation 이나 dense simulation 을 하지 않고, 중요한 material-space sample 과 compact solver anchors 사이를 projection 으로 연결한다.

9 Module 5: Gaussian-to-Proxy Binding

Input

$$G^0, \quad M_{\text{sim}}^0. \quad (45)$$

입력 G^0 는 rendering primitive 인 Gaussian set 이고, M_{sim}^0 는 dynamics 를 담당하는 proxy mesh 이다. 이 모듈은 두 representation 사이의 persistent correspondence 를 만든다. 한 번 binding 이 만들어지면 runtime 에는 Gaussian 이 매 frame 새 triangle 을 찾지 않고, 저장된 material coordinate 를 따라간다.

Operation

각 Gaussian 을 가장 안정적인 mesh triangle 에 binding 한다.

$$B(j) = \arg \min_{T \in F} E_{jT}. \quad (46)$$

$B(j)$ 는 Gaussian j 가 어떤 triangle T 에 붙을지를 결정하는 mapping 이다. 가장 가까운 triangle 만 고르면 surface 반대편이나 noisy mesh 에 잘못 붙을 수 있으므로, distance 뿐 아니라 normal consistency 와 triangle quality 를 함께 본다.

binding energy 예시는 다음과 같다.

$$E_{jT} = \frac{d(\mu_j^0, T)^2}{\sigma_j^2} + \lambda_n(1 - |\langle n_j^G, n_T \rangle|) + \lambda_q Q_T. \quad (47)$$

$d(\mu_j^0, T)$ 는 Gaussian center 와 triangle 사이의 거리이고, σ_j 는 Gaussian scale 기반 normalization 이다. 두 번째 항은 Gaussian normal candidate n_j^G 와 triangle normal n_T 의 정렬도를 본다. 절댓값을 쓰는 이유는 covariance 에서 얻은 eigenvector normal 의 부호가 모호하기 때문이다. Q_T 는 skinny triangle, boundary artifact, low-quality face 에 대한 penalty 로 둘 수 있다.

각 Gaussian 에 대해 저장한다.

$$\mathcal{B}_j = (T_j, b_j, R_{T_j}^0, \delta_j). \quad (48)$$

여기서 $T_j = (a, b, c)$, $b_j = (b_{ja}, b_{jb}, b_{jc})$ 는 barycentric coordinate, $R_{T_j}^0$ 는 canonical triangle local frame, δ_j 는 optional local offset 이다. b_j 는 Gaussian center 가 triangle 위의 어느 material position 에 붙는지를 나타내고, $R_{T_j}^0$ 는 triangle 의 rest local frame 이다. δ_j 를 두면 Gaussian 이 surface 위에 정확히 놓이지 않고 약간 떠 있거나 두께 방향 offset 을 갖는 경우도 표현할 수 있다. 이 정보가 있어야 뒤에서 deformed triangle frame 을 이용해 mean 과 covariance 를 함께 transport 할 수 있다.

Output

$$\{\mathcal{B}_j\}_{j=1}^{N_G}. \quad (49)$$

출력은 Gaussian 마다 하나씩 저장된 binding table 이다. 이 table 은 runtime 동안 고정되며, Gaussian index j 와 proxy triangle/material coordinate 사이의 correspondence 를 보장한다.

Effect

persistent binding 은 mesh deformation 을 Gaussian rendering representation 으로 되돌리기 위한 핵심이다. 이 단계가 있어야 per-frame closest triangle search 나 rebinding 없이 temporal consistency 를 유지할 수 있다. 또한 Gaussian mean, anisotropic covariance, local frame, SH feature 를 mesh deformation 에 따라 일관되게 transport 할 수 있다. 논문 관점에서는 이 부분이 “3DGS 와 simulation proxy 사이의 bridge” 를 이루는 핵심 구성요소이다. Binding 이 불안정하면 solver 가 아무리 좋아도 rendering 단계에서 Gaussian 이 튀거나 surface 에서 미끄러지는 artifact 가 생긴다.

10 Module 6: Blender Dense Supervision Generation

Input

Blender 또는 physics engine 의 known mesh, material, wind condition, camera views.

$$M_{hi}^0, \quad W(x, t), \quad \Pi_t. \quad (50)$$

M_{hi}^0 는 high-resolution reference mesh 또는 simulator-native asset 이다. 이 모듈의 입력은 실제 target 3DGS asset 이라기보다, 학습과 평가를 위해 ground-truth force/deformation/rendering 을 얻을 수 있는 synthetic setup 이다. $W(x, t)$ 는 simulator 에 넣는 wind condition 이고, Π_t 는 rendered reference 를 만들 camera sequence 이다.

Operation

각 asset 에 대해 여러 wind condition 을 simulation 하고, dense state 와 rendered reference 를 저장한다.

$$\mathcal{D}_{\text{Blender}} = \{x_r^{\text{dense}}(t), f_r^{\text{dense}}(t), W(t), I_t^{\text{gt}}\}. \quad (51)$$

$x_r^{\text{dense}}(t)$ 는 dense mesh vertex 또는 sampled point 의 trajectory 이고, $f_r^{\text{dense}}(t)$ 는 같은 위치에서의 reference force 이다. $W(t)$ 는 해당 sequence 의 wind parameter 또는 field sample 이고, I_t^{gt} 는 camera Π_t 에서 렌더링한 reference image 이다. 이 데이터는 runtime 에 직접 쓰기 위한 것이 아니라, force predictor 와 projection module 을 검증하거나 학습하기 위한 supervision 이다.

Output

$$\mathcal{D}_{\text{Blender}}. \quad (52)$$

출력 dataset 은 dense physical signal 과 image-space signal 을 함께 갖는다. Dense force 는 load prediction/projection 의 target 으로 쓰고, rendered image 는 최종 visual response 가 reference 와 얼마나 맞는지 평가하는 데 쓴다.

Effect

Blender supervision 은 runtime engine 이 아니라 학습 데이터와 evaluation ground truth 를 제공한다. 이 데이터는 force predictor, dense-to-proxy projection, SH residual, ablation evaluation 에 사용된다. 즉 이 모듈은 논문 시스템의 runtime dependency 가 아니다. Asset 이 실제 captured 3DGS 뿐인 경우에는 이 모듈을 건너뛰고 analytic force baseline 부터 사용할 수 있으며, synthetic benchmark 를 만들 때만 dense supervision 을 생성한다.

11 Module 7: Conservation-Preserving Dense-to-Proxy Force Projection

Input

Blender dense force 와 proxy/load sample layout.

$$\{f_r^w\}_{r \in \Omega}, \quad \{x_i\}_{i \in \Omega}. \quad (53)$$

Ω 는 dense reference force 를 하나의 proxy patch 또는 load-sampling region 으로 모은 영역이다. f_r^w 는 wind condition w 에서 dense sample r 에 작용한 reference force 이고, x_i 는 그 patch 를 대표하는 proxy/load sample positions 이다. 이 모듈의 목적은 dense supervision 을 sparse proxy representation 으로 줄이되, 전체 force 와 torque 정보를 잃지 않는 것이다.

Operation

patch Ω 에서 dense net force 와 wrench 를 계산한다.

$$F_{\Omega, w}^{\text{dense}} = \sum_{r \in \Omega} f_r^w, \quad (54)$$

$$T_{\Omega, w}^{\text{dense}} = \sum_{r \in \Omega} (x_r - c_\Omega) \times f_r^w. \quad (55)$$

$F_{\Omega,w}^{\text{dense}}$ 는 patch 전체에 작용하는 net force 이고, $T_{\Omega,w}^{\text{dense}}$ 는 patch center c_Ω 기준의 torque 또는 wrench moment 이다. 단순히 force magnitude 만 맞추면 patch 가 회전하려는 경향을 잃을 수 있으므로, torque 보존도 함께 둔다.

raw proxy force $F_{i,w}^{\text{raw}}$ 를 얻은 뒤, conservation-preserving correction 을 푼다.

$$\min_{\{F_{i,w}^*, \tau_{i,w}^*\}} \sum_i \|F_{i,w}^* - F_{i,w}^{\text{raw}}\|^2 + \gamma \|\tau_{i,w}^* - \tau_{i,w}^{\text{raw}}\|^2, \quad (56)$$

subject to:

$$\sum_i F_{i,w}^* = F_{\Omega,w}^{\text{dense}}, \quad (57)$$

$$\sum_i (x_i - c_\Omega) \times F_{i,w}^* + \tau_{i,w}^* = T_{\Omega,w}^{\text{dense}}. \quad (58)$$

$F_{i,w}^{\text{raw}}$ 는 nearest assignment, kernel interpolation, or coarse predictor 로 얻은 초기 sparse force 이다. 최적화는 이 raw force 를 가능한 적게 바꾸면서, patch 전체의 dense net force 와 torque 가 정확히 맞도록 보정한다. $\tau_{i,w}^*$ 는 force 만으로 torque 를 완전히 맞추기 어려울 때 남는 local torque term 으로 볼 수 있다. 이 step 은 sparse representation 이 physical aggregate 를 보존한다는 근거가 된다.

projection risk 는 initial coarse load-sampling layout 또는 patch partition 에서 계산한 correction ratio 로 정의한다.

$$\epsilon_{\Omega,w}^{\text{proj}} = \frac{\sum_i \|F_{i,w}^* - F_{i,w}^{\text{raw}}\|^2 + \gamma \sum_i \|\tau_{i,w}^* - \tau_{i,w}^{\text{raw}}\|^2}{\sum_i \|F_{i,w}^{\text{raw}}\|^2 + \gamma \sum_i \|\tau_{i,w}^{\text{raw}}\|^2 + \varepsilon}. \quad (59)$$

$\epsilon_{\Omega,w}^{\text{proj}}$ 가 크다는 것은 coarse layout 이 dense force 를 잘 표현하지 못해 correction 이 많이 필요했다는 뜻이다. 따라서 이 값은 어디에 load sample 을 더 두어야 하는지 알려주는 diagnostic signal 이다. 분모의 ε 는 raw force 가 거의 0 인 영역에서 ratio 가 폭주하는 것을 막는다.

$$F_{\text{projection_risk}}(p) = \text{normalize} \left(\max_{w \in \mathcal{W}} \epsilon_{\Omega(p),w}^{\text{proj}} \right). \quad (60)$$

이 값은 runtime adaptive sampling 이 아니다. Preprocessing 에서 coarse layout 위에 한 번 계산하고, fixed load sample placement 를 한 번 refine 하는 데 사용한다. $\max_{w \in \mathcal{W}}$ 를 쓰는 이유는 여러 wind condition 중 하나라도 표현하기 어려운 영역이면 load sample refinement 후보로 잡기 위해서이다. 즉 projection risk 는 time-varying anchor 가 아니라 fixed layout 을 더 잘 설계하기 위한 one-time preprocessing score 이다.

$$s_{\text{load}}(p) = w_E E_{\text{prior}}(p) + w_F F_{\text{projection_risk}}(p) + w_R R_G(p) + w_K K_{\text{geom}}(p). \quad (61)$$

$s_{\text{load}}(p)$ 는 load sample density 를 정하는 score 이다. E_{prior} 는 wind exposure 나 silhouette-like prior, $F_{\text{projection_risk}}$ 는 dense force 보존 난이도, R_G 는 Gaussian transport/rendering demand, K_{geom} 는 geometry detail demand 를 나타낸다. 이 score 는 solver mesh resolution 과 별개로 load sample placement 를 조절한다.

Blender-supervised MVP 에서는 다음처럼 시작할 수 있다.

$$s_{\text{load}}^{\text{Blender-MVP}}(p) = 1.2 F_{\text{projection_risk}}(p) + 0.5 K_{\text{geom}}(p). \quad (62)$$

초기에는 Blender 에서 얻은 projection risk 와 geometry curvature 만으로도 어느 영역의 load sampling 이 부족한지 확인할 수 있다. Blender supervision 이 없는 실제 captured asset 에서는 analytic exposure prior 와 geometry/Gaussian demand 로 대체한다.

Output

$$F_i^*, \quad \tau_i^*, \quad F_{\text{projection_risk}}(p), \quad Q_{\text{load}}^{\text{refined}}. \quad (63)$$

출력은 보정된 sparse force/torque 와, 그 보정량으로부터 얻은 projection risk field, 그리고 한 번 refine 된 fixed load sample set 이다. $Q_{\text{load}}^{\text{refined}}$ 도 runtime 에는 고정된다.

Effect

이 모듈은 dense Blender force 를 coarse proxy/load samples 로 압축하면서 net force 와 wrench 를 보존한다. 따라서 sparse proxy simulation 이 단순 계산 shortcut 이 아니라, dense force supervision 을 보존하는 representation 으로 정당화된다. 논문 실험에서는 이 모듈을 통해 random/coarse sampling 대비 projection-risk-aware load sampling 이 force preservation 과 visual response 를 개선하는지 ablation 할 수 있다.

12 Module 8: Runtime Wind Field Evaluation

Input

$$M(t), \quad W(x, t), \quad Q_{\text{load}}. \quad (64)$$

$M(t)$ 는 현재 frame 의 deformed proxy mesh 이고, $W(x, t)$ 는 사용자나 procedural controller 가 정의한 wind velocity field 이다. Q_{load} 는 Module 4 또는 7 에서 만든 fixed material-space load samples 이다. 이 모듈은 mesh topology 나 load sample placement 를 바꾸지 않고, 현재 위치에서 wind 를 평가하는 runtime query 단계이다.

Operation

wind field 는 lightweight analytic field 로 둔다.

$$W(x, t) = W_{\text{global}}(t) + \sum_k W_{\text{source},k}(x, t) + W_{\text{gust}}(x, t). \quad (65)$$

$W_{\text{global}}(t)$ 는 전체 장면에 적용되는 direction/strength 이며, fan 이나 directional wind control 로 볼 수 있다. $W_{\text{source},k}(x, t)$ 는 위치 의존적인 local source, 예를 들어 fan, vortex, brush stroke wind 를 표현한다. $W_{\text{gust}}(x, t)$ 는 sinusoidal gust 나 turbulence noise 처럼 시간적으로 흔들리는 성분이다. 이 설계는 full CFD 를 복원하려는 것이 아니라, controllable one-way wind input 을 제공하는 데 목적이 있다.

각 load sample 위치와 velocity 를 계산한다.

$$x_l(t) = \sum_{i \in T_l} b_{li} x_i(t), \quad (66)$$

$$v_l(t) = \sum_{i \in T_l} b_{li} v_i(t), \quad (67)$$

$$W_l(t) = W(x_l(t), t). \quad (68)$$

$x_l(t)$ 와 $v_l(t)$ 는 load sample $q_l = (T_l, b_l)$ 이 현재 deformed mesh 에서 어디에 있고 얼마나 움직이는지 나타낸다. 둘 다 barycentric interpolation 으로 계산하므로, sample 은 같은 material point 를 계속 따라간다. $W_l(t)$ 는 그 현재 위치에서 wind field 를 query 한 값이다. 이 단계까지는 wind 와 object motion 을 별도로 계산하며, 실제 상대 바람은 다음 exposure module 에서 만든다.

Output

$$W_l(t), \quad x_l(t), \quad v_l(t). \quad (69)$$

출력은 load sample 별 current position, current material velocity, local wind velocity 이다. 이 세 값은 aerodynamic force 를 계산하기 위한 최소 runtime feature 이다.

Effect

user wind input 을 proxy surface 의 local wind 로 변환한다. Full CFD 를 주장하지 않으면서 controllable wind editing 을 가능하게 한다. 사용자가 wind direction, gust frequency, fan source 를 조절하면 이 모듈의 $W_l(t)$ 가 바뀌고, 이후 force 와 deformation response 가 달라진다.

13 Module 9: Wind Exposure Estimation

Input

$$G(t) \text{ or } M(t), \quad \hat{w}(t), \quad Q_{\text{load}}. \quad (70)$$

입력은 현재 deformed Gaussian state 또는 proxy mesh, wind direction $\hat{w}(t)$, 그리고 fixed load samples 이다. Exposure estimation 은 단순히 wind magnitude 를 쓰는 것이 아니라, 바람 방향에서 보았을 때 각 load sample 이 얼마나 직접적으로 바람을 맞는지 추정한다. front surface, occluded surface, backside surface 가 같은 force 를 받지 않도록 하기 위한 단계이다.

Operation

current Gaussian or proxy state 를 wind-axis coordinate 에서 rasterize 하여 transmittance map 을 만든다.

$$S_t(u) = \exp \left(- \int \alpha_t(s) ds \right). \quad (71)$$

u 는 wind direction 에 수직인 2D wind-axis image coordinate 이고, s 는 wind ray 를 따라가는 depth coordinate 이다. $\alpha_t(s)$ 는 그 ray 위에서 만나는 opacity density 이다. $S_t(u)$ 는 바람이 해당 ray 를 따라 들어올 때 얼마나 통과하는지를 나타내는 transmittance 이다. 값이 1 에 가까우면 wind-facing exposure 가 크고, 0 에 가까우면 앞쪽 surface 에 의해 막혀 있다고 본다.

Gaussian splatting approximation 은 다음과 같다.

$$S_t(u) \approx \prod_j (1 - \alpha_j K_j^w(u)). \quad (72)$$

$K_j^w(u)$ 는 Gaussian j 를 wind-axis plane 에 projection 했을 때의 footprint kernel 이다. 이 approximation 은 3DGS 의 alpha compositing 직관을 wind direction 으로 돌려 쓴 것이다. 정확한 fluid occlusion 은 아니지만, self-shadowing 과 wind blocking 을 가볍게 feature 화할 수 있다.

각 fixed load sample 에서 exposure 를 sampling 한다.

$$e_l(t) = \text{sample}(S_t, \pi_w(x_l(t))). \quad (73)$$

π_w 는 3D point 를 wind-axis plane 으로 projection 하는 함수이다. 따라서 $e_l(t)$ 는 load sample l 이 현재 frame 에서 얼마나 wind-exposed 인지 나타내는 scalar 이다. 중요하게도 sample 위치 $x_l(t)$ 는 움직이지만 sample identity q_l 은 고정되어 있다.

relative effective wind 는:

$$u_l^{\text{eff}}(t) = e_l(t)W_l(t) - v_l(t). \quad (74)$$

$u_l^{\text{eff}}(t)$ 는 object material point 가 실제로 느끼는 상대 바람이다. $e_l(t)W_l(t)$ 는 exposure 로 감쇠된 wind velocity 이고, $v_l(t)$ 는 material point 자체의 velocity 이다. 물체가 wind 방향으로 같이 움직이면 상대 wind 가 줄고, 반대로 움직이면 더 큰 force 가 생긴다.

Output

$$e_l(t), \quad u_l^{\text{eff}}(t). \quad (75)$$

출력은 force predictor 가 바로 사용할 exposure scalar 와 relative wind vector 이다. $e_l(t)$ 는 load magnitude 뿐 아니라 SH residual 이나 appearance correction 의 feature 로도 사용할 수 있다.

Effect

exposure 는 self-shadowing, wind blocking, front/back visibility 를 force feature 로 반영한다. 중요한 점은 exposure 가 frame 마다 변해도 anchor 나 load sample 을 재배치하지 않는다는 것이다. Fixed material-space sample 에서 값만 갱신한다. 이 설계는 사용자가 제기했던 “wind field 가 frame 마다 바뀌면 anchor 도 바뀌어야 하는가” 라는 문제를 피한다. Anchor 와 load sample 은 고정하고, 그 위에서 exposure value 만 time-varying field 로 평가한다.

14 Module 10: Force-Space Load Prediction

Input

$$z_l(t), \quad u_l^{\text{eff}}(t), \quad e_l(t), \quad \theta_l. \quad (76)$$

$z_l(t)$ 는 load sample 주변의 local geometry/state feature 이다. 예를 들어 normal, area, curvature, strain, previous velocity, local frame 등을 포함할 수 있다. $u_l^{\text{eff}}(t)$ 와 $e_l(t)$ 는 wind-related feature 이고, θ_l 은 material parameter 또는 profile preset 에서 온 local material feature 이다.

Operation

deformation state 를 직접 예측하지 않고 aerodynamic load 를 예측한다.

$$(f_l(t), \tau_l(t)) = P_\theta(z_l(t), u_l^{\text{eff}}(t), e_l(t), \theta_l). \quad (77)$$

P_θ 는 analytic formula, learned network, or analytic-plus-residual model 일 수 있다. 출력 $f_l(t)$ 는 load sample 에 작용하는 force 이고, $\tau_l(t)$ 는 필요할 경우 local torque 또는 bending moment 이다. 우리가 deformation 을 직접 regression 하지 않는 이유는 solver 가 topology 와 constraints 를 처리하도록 남겨두기 위해서이다.

analytic drag baseline 은 다음처럼 둘 수 있다.

$$f_l^{\text{drag}}(t) = \frac{1}{2} \rho_{\text{air}} C_D A_l e_l(t) \|u_l^{\text{eff}}(t)\|^2 \cdot \max(0, \langle n_l(t), \hat{u}_l(t) \rangle) n_l(t). \quad (78)$$

ρ_{air} 는 air density, C_D 는 drag coefficient, A_l 은 load sample 이 대표하는 area 이다. $\max(0, \langle n_l, \hat{u}_l \rangle)$ 는 wind-facing surface 만 force 를 받도록 하는 orientation gate 이다. 이 baseline 은 물리적으로 완전한 aerodynamics 는 아니지만, wind strength 가 제공에 비례하고 surface orientation 에 의존한다는 기본 경향을 제공한다.

학습형 force predictor 는 residual 형태가 안전하다.

$$f_l(t) = f_l^{\text{analytic}}(t) + \Delta f_l^\theta(t). \quad (79)$$

Residual 형태는 analytic baseline 이 설명하지 못하는 flutter, vortex-like response, material-specific correction 만 학습하게 한다. 이렇게 하면 network 가 전체 dynamics 를 외우는 것보다 wind control 과 out-of-distribution amplitude 에 더 안정적일 가능성이 높다.

Output

$$f_l(t), \quad \tau_l(t). \quad (80)$$

출력은 아직 solver vertex force 가 아니라 load sample force 이다. 다음 Module 11 에서 barycentric projection 을 통해 simulation anchors 로 누적된다.

Effect

이 모듈은 direct deformation regression 을 피하고, wind amplitude/direction/frequency 에 대해 더 제어 가능한 response 를 만든다. 또한 Blender dense force supervision 과 자연스럽게 연결된다. 논문 framing 에서는 “learned deformation” 보다 “force-space wind response prior” 에 가깝게 설명하는 편이 안전하다.

15 Module 11: Load-to-Anchor Force Projection

Input

$$f_l(t), \quad \tau_l(t), \quad q_l = (T_l, b_l). \quad (81)$$

입력 force 는 load sample 위치에서 계산된 값이고, XPBD solver 는 vertex anchor force 를 필요로 한다. 따라서 이 모듈은 material-space load sample 과 simulation anchor 사이의 force transfer 를 담당한다. $q_l = (T_l, b_l)$ 가 있기 때문에 sample force 를 어떤 triangle 의 어떤 vertex 들로 보낼지 알 수 있다.

Operation

기본적으로 barycentric distribution 을 사용한다.

$$f_i(t) += b_{li} f_l(t), \quad i \in T_l. \quad (82)$$

이 식은 load sample force 를 triangle vertices 에 barycentric weight 로 분배한다. Sample 이 vertex 에 가까우면 해당 vertex 가 더 큰 force 를 받고, triangle 중앙이면 세 vertex 가 비슷하게 나눠 받는다. 이 방식은 간단하지만 material coordinate 와 잘 맞고, total force 보존도 직관적이다.

external force 는 다음과 같다.

$$f_i^{\text{ext}}(t) = \sum_{l:i \in T_l} b_{li} f_l(t) + f_i^{\text{gravity}} + f_i^{\text{user}}. \quad (83)$$

$f_i^{\text{ext}}(t)$ 는 solver 가 다음 step 을 예측할 때 사용하는 외력이다. wind load 외에도 gravity, user handle, scripted interaction force 를 같은 형태로 합산할 수 있다. Torque $\tau_l(t)$ 를 사용하는 경우에는 equivalent force couple 이나 bending constraint target 으로 변환하는 확장이 가능하다.

Output

$$f_i^{\text{ext}}(t). \quad (84)$$

출력은 simulation anchor 별 external force field 이다. 이제 force 는 load sample domain 이 아니라 solver vertex domain 에 놓인다.

Effect

force sampling result 를 XPBD solver 가 사용할 vertex force 로 변환한다. 이 단계는 fixed load samples 와 fixed simulation anchors 를 연결한다. 여기서도 topology 나 binding 은 바뀌지 않는다. Frame 마다 바뀌는 것은 $f_l(t)$ 와 $f_i^{\text{ext}}(t)$ 의 값뿐이다.

16 Module 12: Proxy Dynamics Solver

Input

$$x_i(t), \quad v_i(t), \quad f_i^{\text{ext}}(t), \quad m_i, \quad \theta_i, \quad C. \quad (85)$$

입력은 현재 proxy vertex state 와 외력, mass/material parameter, constraint set 이다. C 는 stretch, bending, pin, attachment 같은 position-based constraints 의 집합이다. 이 모듈은 Gaussian 을 직접 움직이지 않고, proxy mesh 의 physically plausible deformation 을 먼저 계산한다.

Operation

먼저 external force 를 이용해 predicted velocity 와 position 을 계산한다.

$$v_i^* = v_i(t) + \Delta t \frac{f_i^{\text{ext}}(t)}{m_i}, \quad (86)$$

$$x_i^* = x_i(t) + \Delta t v_i^*. \quad (87)$$

v_i^* 와 x_i^* 는 constraint projection 전의 tentative state 이다. 외력은 이 단계에서 motion 을 예측하고, 이후 XPBD constraints 가 stretch, bending, pin 조건을 만족하도록 위치를 보정한다.
stretch constraint:

$$C_{ik}^{\text{stretch}}(x) = \|x_i - x_k\| - \ell_{ik}^0, \quad \ell_{ik}^0 = \|x_i^0 - x_k^0\|. \quad (88)$$

Stretch constraint 는 edge length 가 rest length ℓ_{ik}^0 에서 너무 벗어나지 않게 한다. Cloth 나 paper-like object 에서 면이 비현실적으로 늘어나는 것을 막는 기본 constraint 이다.

bending constraint:

$$C_e^{\text{bend}}(x) = \theta_e(x) - \theta_e^0. \quad (89)$$

Bending constraint 는 adjacent faces 사이의 dihedral angle 을 rest angle θ_e^0 근처로 유지한다. 이 향이 약하면 천처럼 잘 휘고, 강하면 종이비행기나 연의 rib 처럼 형태를 유지한다.

pin constraint:

$$C_i^{\text{pin}}(x) = x_i - x_i^{\text{target}}. \quad (90)$$

Pin constraint 는 support, handle, attachment point 를 target position 에 묶는다. Hard pin 과 soft pin 의 차이는 이 constraint 의 compliance 를 0 에 가깝게 둘지, 유한한 값으로 둘지에 있다.

XPBD update for a generic constraint:

$$\Delta\lambda = -\frac{C(x) + \tilde{\alpha}\lambda}{\sum_i w_i \|\nabla_{x_i} C\|^2 + \tilde{\alpha}}, \quad (91)$$

$$x_i \leftarrow x_i + w_i \nabla_{x_i} C \Delta\lambda, \quad (92)$$

λ 는 constraint multiplier 이고, $\Delta\lambda$ 는 이번 projection iteration 에서 추가되는 correction 이다. $\nabla_{x_i} C$ 는 constraint 를 줄이기 위해 vertex i 를 어느 방향으로 움직여야 하는지 알려준다. XPBD 는 compliance α 를 통해 stiffness 를 time step 과 iteration 수에 덜 민감하게 다룰 수 있어 interactive simulation 에 적합하다.

where:

$$w_i = \frac{1}{m_i}, \quad \tilde{\alpha} = \frac{\alpha}{\Delta t^2}. \quad (93)$$

w_i 는 inverse mass 이므로 무거운 vertex 는 덜 움직이고, 가벼운 vertex 는 더 많이 correction 된다. $\tilde{\alpha}$ 는 time-step-scaled compliance 이며, $\alpha = 0$ 이면 거의 hard constraint 처럼 동작한다.

velocity update:

$$v_i(t+1) = \frac{x_i(t+1) - x_i(t)}{\Delta t}. \quad (94)$$

Constraint projection 이 끝난 뒤 위치 차분으로 velocity 를 갱신한다. 필요하다면 velocity damping 이나 constraint-aware velocity correction 을 추가할 수 있지만, MVP 에서는 이 기본 update 로 시작한다.

Output

$$M(t+1) = (V(t+1), E, F). \quad (95)$$

출력은 다음 frame 의 deformed proxy mesh 이다. Topology E, F 는 그대로이고 vertex positions $V(t+1)$ 만 바뀐다.

Effect

connected surface deformation, boundary constraints, material stiffness, damping, and wind-induced force propagation 을 처리한다. Gaussian 을 직접 particle 로 움직이는 것보다 topology-aware response 를 만들 수 있다. 이 모듈 덕분에 wind response 는 개별 Gaussian 의 독립 이동이 아니라, mesh connectivity 와 material constraints 를 따른 deformation 으로 표현된다. 다음 Module 13 은 이 proxy deformation 을 다시 Gaussian rendering representation 으로 전달한다.

17 Module 13: Gaussian Attribute Transport

Input

$$M(t+1), \quad \mathcal{B}_j = (T_j, b_j, R_{T_j}^0, \delta_j), \quad g_j^0. \quad (96)$$

입력은 새로 계산된 deformed proxy, Module 5 에서 저장한 Gaussian binding, 그리고 canonical Gaussian state 이다. 이 모듈은 simulation proxy 와 rendering 3DGS 를 다시 연결하는 runtime bridge 이다. Binding 이 고정되어 있기 때문에 각 Gaussian 은 자신이 붙은 triangle 의 현재 frame 을 통해 업데이트된다.

Operation

triangle $T_j = (a, b, c)$ 의 current local frame $R_{T_j}(t+1)$ 을 계산한다. Relative rotation 은:

$$\Delta R_{T_j}(t+1) = R_{T_j}(t+1)(R_{T_j}^0)^\top. \quad (97)$$

$R_{T_j}^0$ 는 rest triangle frame 이고, $R_{T_j}(t+1)$ 는 deformed triangle frame 이다. ΔR_{T_j} 는 rest frame 에서 current frame 으로 가는 local rotation 이다. 이 rotation 이 있어야 Gaussian ellipsoid 가 surface normal/tangent 변화에 맞춰 함께 회전한다.

Gaussian center update:

$$\mu_j(t+1) = b_{ja}x_a(t+1) + b_{jb}x_b(t+1) + b_{jc}x_c(t+1) + R_{T_j}(t+1)\delta_j. \quad (98)$$

앞의 세 항은 barycentric coordinate 로 triangle 위 material point 를 따라가는 위치이다. $R_{T_j}(t+1)\delta_j$ 는 optional local offset 을 current frame 방향으로 회전시켜 더한다. 따라서 Gaussian 이 surface 위 또는 근처의 같은 material 위치를 따라 움직인다.

covariance transport:

$$\Sigma_j(t+1) = \Delta R_{T_j}(t+1)\Sigma_j^0\Delta R_{T_j}(t+1)^\top. \quad (99)$$

이 식은 Gaussian center 만 translation 하는 것이 아니라 anisotropic covariance frame 도 함께 transport 한다. 즉 leaf 나 cloth surface 가 회전하면 Gaussian ellipsoid 의 얇은 normal 방향도 함께 회전한다. 이 처리가 빠지면 위치는 따라가더라도 splat orientation 이 canonical 상태에 남아 surface 와 어긋나는 artifact 가 생길 수 있다.

선택적으로 face area stretch 를 이용한 scale correction 을 사용할 수 있다.

$$a_T(t) = \frac{\text{area}(T(t))}{\text{area}(T^0)}. \quad (100)$$

$a_T(t)$ 는 triangle area 가 rest 대비 얼마나 늘거나 줄었는지를 나타낸다. 이를 이용하면 surface stretch 에 따라 Gaussian scale 을 약간 키우거나 줄일 수 있다. 다만 scale 을 직접 바꾸면 splat size popping 이나 density 변화가 생길 수 있으므로, 초기 구현에서는 보수적으로 다룬다.

단, 초기 구현에서는 splat popping 을 피하기 위해 center 와 covariance rotation 만 먼저 검증하는 것이 안전하다.

Output

$$\mu_j(t+1), \quad \Sigma_j(t+1), \quad R_j(t+1), \quad s_j(t+1). \quad (101)$$

Renderer 가 covariance matrix 를 직접 받지 않고 rotation/scale parameter 를 요구하는 경우, $\Sigma_j(t+1)$ 를 다시 eigen-decomposition 하거나 transported frame 과 scale 로 분해해 $R_j(t+1), s_j(t+1)$ 를 만든다. Opacity 와 appearance 는 기본적으로 canonical 값을 유지하고, 필요한 correction 은 다음 Module 14 에서 처리한다.

Effect

mesh deformation 을 다시 3DGS rendering representation 으로 변환한다. 이는 position-only update 가 아니라 mean, anisotropic covariance, local frame 을 포함하는 Gaussian attribute transport 이다. 이 모듈이 논문의 persistent representation bridge 에서 가장 직접적인 runtime 단계이다. 사용자 관점에서는 proxy mesh 를 보지 않지만, Gaussian rendering 결과는 이 transport 를 통해 wind-driven deformation 을 반영한다.

18 Module 14: SH / Appearance Correction

Input

$$c_j^0, \Delta T_j(t), \nabla x_{T_j}(t), e_{T_j}(t), W_{T_j}(t), \theta_{T_j}. \quad (102)$$

c_j^0 는 canonical SH 또는 appearance coefficients 이다. $\Delta T_j(t)$ 는 Gaussian 이 붙은 triangle/local frame 의 변화, $\nabla x_{T_j}(t)$ 는 local deformation gradient 나 strain-like feature 이다. $e_{T_j}(t)$ 와 $W_{T_j}(t)$ 는 exposure 와 wind feature 이고, θ_{T_j} 는 material feature 이다. 이 모듈은 geometry transport 만으로 설명되지 않는 view-dependent appearance 변화를 보정한다.

Operation

canonical SH 를 local frame 변화에 맞게 analytic correction 하고, residual 만 예측한다.

$$c_j(t) = R_{SH}(\Delta T_j(t))c_j^0 + \Delta c_j(t). \quad (103)$$

$R_{SH}(\Delta T_j(t))$ 는 local frame rotation 이 SH coefficients 에 미치는 analytic rotation 이다. Geometry 가 회전하면 view-dependent lobes 도 같은 방향으로 회전해야 하므로, 이 부분은 가능하면 closed-form correction 으로 처리한다. $\Delta c_j(t)$ 는 analytic rotation 만으로 설명되지 않는 shading, thickness, exposure, deformation-induced appearance residual 이다.

residual predictor:

$$\Delta c_j(t) = Q_\psi(c_j^0, \Delta T_j(t), \nabla x_{T_j}(t), e_{T_j}(t), W_{T_j}(t), \theta_{T_j}). \quad (104)$$

Q_ψ 는 작은 MLP 나 lightweight module 로 둘 수 있다. 입력 feature 가 local binding/frame 기준으로 정의되면, asset 전체 좌표계가 바뀌어도 residual 이 더 안정적으로 동작한다. 중요한 점은 SH residual 이 geometry/dynamics 를 대체하지 않고, runtime rendering fidelity 를 보완한다는 것이다.

Output

$$c_j(t). \quad (105)$$

출력은 현재 frame 에서 renderer 에 넣을 appearance coefficients 이다. MVP 에서는 $\Delta c_j(t) = 0$ 으로 두고 analytic SH rotation 만 검증하거나, degree-0 color 는 고정하고 higher-order SH 만 조정하는 단계적 구현도 가능하다.

Effect

이 모듈은 deformation-aware appearance fidelity 를 보완한다. 하지만 headline contribution 이 아니라, geometry/dynamics pipeline 을 완성하기 위한 supporting module 로 두는 것이 좋다. 특히 runtime 구동을 위해 SH residual 을 남겨두면, geometry transport 후에도 남는 view-dependent artifact 를 가볍게 흡수할 수 있다. 다만 이 모듈의 성능을 주장하려면 geometry-only, analytic SH rotation only, residual 추가의 ablation 이 필요하다.

19 Module 15: 3DGS Rendering and Response Output

Input

$$G(t) = \{\mu_j(t), \Sigma_j(t), \alpha_j(t), c_j(t)\}, \quad \Pi_t. \quad (106)$$

$G(t)$ 는 모든 runtime update 가 반영된 dynamic Gaussian set 이다. 각 Gaussian 은 transported mean/covariance, opacity, corrected appearance 를 갖는다. Π_t 는 현재

frame 의 camera intrinsics/extrinsics 이며, preview 에서는 fixed camera 나 turntable path, evaluation 에서는 ground-truth camera path 가 될 수 있다.

Operation

standard 3DGS renderer 를 사용한다.

$$I_t = \mathcal{R}_{3DGS}(G(t), \Pi_t). \quad (107)$$

이 단계에서는 새로운 renderer 를 발명하는 것이 아니라, updated Gaussian state 를 기존 3DGS renderer 에 넣는다. Renderer backend 는 gsplat, diff-gaussian-rasterization, or CPU/debug renderer 일 수 있지만 논리적으로는 같은 함수 $\mathcal{R}_{3DGS}$ 로 본다. 따라서 contribution 은 rendering algorithm 자체보다, renderer 에 들어가기 전 dynamic $G(t)$ 를 어떻게 안정적으로 만드는데 있다.

Output

$$\boxed{\mathcal{I} = \{I_t\}_{t=1}^T} \quad \text{or} \quad \boxed{I_t}. \quad (108)$$

단일 frame rendering 이면 I_t 가 출력이고, animation 이나 simulation preview 이면 frame sequence $\mathcal{I}$ 가 출력이다. 이 결과가 사용자가 직접 보는 최종 response 이다. export 가능한 dynamic representation 은:

$$\boxed{\{G(t)\}_{t=1}^T}. \quad (109)$$

$\{G(t)\}_{t=1}^T$ 를 저장하면 rendering 결과뿐 아니라 dynamic Gaussian asset 자체를 export 할 수 있다. 이 representation 은 다른 camera path 로 다시 render 하거나, downstream editing/evaluation 에 사용할 수 있다.

Effect

사용자는 최종적으로 high-quality 3DGS novel-view rendering response 를 얻는다. 내부적으로는 mesh proxy 가 simulation 을 담당하지만, 입력과 출력은 여전히 3DGS asset 중심으로 유지된다. 이 점이 pipeline 의 중요한 framing 이다. Mesh proxy 는 visible product 가 아니라 deformation scaffold 이고, 최종 user-facing representation 은 trained 3DGS 와 그 dynamic response 이다.

20 Runtime Loop

Asset setup 과 training 이 완료되어 다음이 준비되어 있다고 가정한다.

$$G^0, \quad M^0, \quad \mathcal{B}, \quad Q_{\text{load}}, \quad P_\theta, \quad Q_\psi. \quad (110)$$

G^0 는 canonical Gaussian asset 이고, M^0 는 fixed-topology simulation proxy 이다. $\mathcal{B}$ 는 Gaussian-to-proxy binding table, Q_{load} 는 fixed material-space load samples 이다. P_θ 와 Q_ψ 는 각각 force predictor 와 optional SH residual predictor 이며, analytic baseline 만 사용할 경우에는 learned predictor 없이도 loop 를 실행할 수 있다. 이 목록은 runtime loop 가 시작되기 전에 asset setup 또는 training/preprocessing 단계에서 준비되어 있어야 하는 persistent state 이다.

각 frame 에서의 runtime loop 는 다음과 같다.

1. Load sample 위치 계산:

$$x_l(t) = \sum_{i \in T_l} b_{li} x_i(t).$$

2. Wind 계산:

$$W_l(t) = W(x_l(t), t).$$

3. Exposure 계산:

$$e_l(t) = \text{sample}(S_t, \pi_w(x_l(t))).$$

4. Effective wind 계산:

$$u_l^{\text{eff}}(t) = e_l(t)W_l(t) - v_l(t).$$

5. Force 예측:

$$(f_l(t), \tau_l(t)) = P_\theta(z_l(t), u_l^{\text{eff}}(t), e_l(t), \theta_l).$$

6. Vertex force 누적:

$$f_i^{\text{ext}}(t) = \sum_{l:i \in T_l} b_{li} f_l(t).$$

7. XPBD/PBD solver:

$$x(t+1) = \Phi_{\text{XPBD}}(x(t), v(t), f^{\text{ext}}(t), C, \theta).$$

8. Gaussian geometry update:

$$\mu_j(t+1) = \sum_{k \in T_j} b_{jk} x_k(t+1) + R_{T_j}(t+1) \delta_j,$$

$$\Sigma_j(t+1) = \Delta R_{T_j}(t+1) \Sigma_j^0 \Delta R_{T_j}(t+1)^\top.$$

9. Gaussian appearance update:

$$c_j(t+1) = R_{\text{SH}}(\Delta T_j(t+1)) c_j^0 + Q_\psi(\dots).$$

10. Render:

$$I_{t+1} = \mathcal{R}_{\text{3DGS}}(G(t+1), \Pi_{t+1}).$$

이 loop 에서 매 frame 바뀌는 것은 wind query, exposure value, load force, proxy vertex positions, transported Gaussian attributes 이다. 반대로 proxy topology, Gaussian binding, simulation anchor index, load sample material coordinate 는 바뀌지 않는다. 이 separation 이 temporal stability 와 realtime implementation 의 핵심이다.

21 Minimum Development Version

구현 복잡도를 낮추려면 다음 최소 버전부터 시작한다.

1. trained 3DGS input 또는 synthetic Gaussians
2. fixed-topology mesh proxy, 처음에는 known mesh 사용 가능
3. Gaussian-to-triangle binding
4. procedural wind load
5. XPBD/PBD mesh deformation
6. Gaussian mean/covariance transport
7. canonical SH 또는 analytic SH correction
8. 3DGS rendering output

이 최소 버전은 논문 최종 시스템의 모든 장치를 한 번에 구현하려는 계획이 아니다. 먼저 “mesh 가 움직이면 Gaussian mean 과 covariance 가 안정적으로 따라가는가” 를 확인하는 loop 이다. 따라서 exposure estimator, dense Blender supervision, learned force predictor, SH residual 은 이 단계에서 없어도 된다.

이 최소 loop 는 다음과 같다.

$$G^0 \rightarrow M^0 \rightarrow \mathcal{B} \rightarrow M(t) \rightarrow G(t) \rightarrow I_t. \quad (111)$$

여기서 $G^0 \rightarrow M^0$ 는 proxy construction, $M^0 \rightarrow \mathcal{B}$ 는 Gaussian binding, $M(t) \rightarrow G(t)$ 는 attribute transport 를 의미한다. 초기 implementation 에서는 known mesh 와 synthetic Gaussians 를 사용해도 좋다. 핵심 성공 기준은 visual plausibility 보다 numeric/structural correctness 이다: binding 이 고정되는가, covariance 가 triangle rotation 을 따라가는가, solver topology 가 유지되는가, renderer 가 updated Gaussian 을 정상적으로 받는가.

나중에 추가할 모듈은 Gaussian-opacity exposure, Blender dense force projection, learned force predictor, SH residual, material parameter calibration, projection-risk-based load sample refinement 이다. 이 순서는 위험도가 낮은 core bridge 부터 확인한 뒤, force realism 과 appearance fidelity 를 점진적으로 올리는 개발 전략이다.

22 Module Hierarchy for Paper Framing

현재 pipeline 은 구성요소가 많기 때문에, 논문에서 모든 것을 같은 수준의 contribution 으로 쓰면 안 된다. 권장 hierarchy 는 다음과이다.

위계	모듈
Main contribution	fixed-topology mesh-proxy scaffold for wind-deformable 3DGS
Main contribution	persistent Gaussian-to-proxy binding and attribute transport
Secondary contribution	force-space wind deformation on proxy, with Blender dense-force supervision
Supporting module	Gaussian-opacity exposure estimator
Supporting module	conservation-preserving dense-to-proxy force projection
Supporting module	deformation-aware SH residual correction
Implementation detail	anchor/load sample scoring, material parameter regularization, projection-risk refinement

이 hierarchy 의 의도는 시스템을 약하게 보이게 하려는 것이 아니라, reviewer 가 핵심 claim 을 놓치지 않게 하는 것이다. Main contribution 은 3DGS 와 deformable proxy 사이의 persistent bridge 이고, secondary/supporting modules 는 그 bridge 를 더 안정적이고 설득력 있게 만드는 장치이다. 특히 SH residual 과 dense-force projection 은 중요하지만, 그것들이 논문의 주장을 대신하지 않도록 서술해야 한다.

23 Design Dependencies

- **Proxy mesh** provides topology, constraints, local frames, and force propagation.
- **Persistent binding** enables stable Gaussian attribute transport without per-frame rebinding.

- **Fixed load samples** allow time-varying wind exposure and force values without changing anchors.
- **Dense-to-proxy projection** compresses Blender dense supervision while preserving net force and wrench.
- **SH residual** improves rendering fidelity but should not dominate the paper narrative.

이 dependency 를 거꾸로 읽으면 구현 순서도 된다. Proxy mesh 와 persistent binding 이 먼저 안정되어야 wind force 나 SH residual 을 얻었을 때 문제가 어디에서 생겼는지 구분할 수 있다. Fixed load samples 는 wind exposure 가 시간에 따라 바뀌어도 correspondence 를 유지하게 해주고, dense-to-proxy projection 은 학습/평가 단계에서 sparse load representation 을 정당화한다.

24 Suggested Coding-Agent Prompt

```
Use the pipeline note `3dgs_wind_response_pipeline_2026-05-12.tex` as the
authoritative module-order reference.
Revise the idea sketch so that the method is presented as a Persistent Mesh
-Proxy Wind Response Pipeline for 3DGS.
Separate asset setup, training/preprocessing, and runtime modules.
Make clear that runtime does not perform remeshing, Gaussian rebinding, or
load-sample relocation.
Use simulation mesh vertices as fixed simulation anchors and fixed material
-space quadrature points as load samples.
Present Gaussian-to-proxy binding as the key bridge for transporting mean,
covariance, local frame, and appearance.
Keep Blender dense-force supervision and conservation-preserving force
projection as supporting/training modules.
Keep SH residual correction as a supporting appearance module, not the
headline contribution.
```

25 Decision Log Entry

2026-05-12 Pipeline Decomposition

Decision. Record the method as the **Persistent Mesh-Proxy Wind Response Pipeline for 3DGS**. The pipeline starts from a trained 3DGS asset, constructs a fixed-topology simulation proxy, sets fixed simulation anchors and fixed material-space load samples, binds Gaussians persistently to proxy triangles, computes wind loads at runtime, solves proxy dynamics, transports Gaussian attributes, and renders dynamic 3DGS frames.

Reason. The current idea sketch contains many modules. Decomposing them by execution phase and dependency clarifies that only a compact subset runs every frame. This also keeps the paper framing focused on the representation bridge rather than presenting every supporting mechanism as a separate contribution.

Next. Rewrite the method section around four high-level blocks: persistent mesh-proxy scaffold, Gaussian-to-proxy binding and attribute transport, force-space proxy wind dynamics, and rendering/appearance correction. Move detailed anchor/load sample scoring and projection-risk refinement to implementation details or supplementary material.

26 One-Paragraph Method Summary

Given a trained 3DGS asset G^0 , we first construct a fixed-topology simulation proxy M^0 and bind each Gaussian to a proxy triangle using persistent barycentric coordinates and a local frame. At runtime, a wind field and Gaussian/proxy-based exposure estimator produce aerodynamic loads on fixed material-space load samples. These loads are projected to simulation anchors and integrated by an XPBD solver to obtain the deformed proxy $M(t)$. The proxy deformation is then transported back to the Gaussian means, covariances, and local frames through the stored bindings. Finally, canonical SH coefficients are analytically corrected and optionally refined by a lightweight residual network before standard 3DGS rendering produces the output frame.

References

- [1] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH*, 1996.
- [2] Antoine Guedon and Vincent Lepetit. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [3] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.
- [4] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics*, 43(6), 2024.